

# Introdução à Programação Orientada a Objetos com Java

Material Didático Universitário

4 de agosto de 2026

## Conteúdo

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Aula 1: Primeiros Passos com Java e POO no IntelliJ IDEA</b> | <b>2</b> |
| 1.1      | O Ecossistema Java . . . . .                                    | 2        |
| 1.2      | Preparando o Ambiente: IntelliJ IDEA . . . . .                  | 2        |
| 1.3      | Anatomia do Seu Primeiro Código . . . . .                       | 2        |
| 1.4      | O que é Programação Orientada a Objetos (POO)? . . . . .        | 3        |
| 1.5      | Exercício de Fixação . . . . .                                  | 3        |
| <b>2</b> | <b>Aula 2: Desvendando a Orientação a Objetos</b>               | <b>4</b> |
| 2.1      | O Paradigma Orientado a Objetos . . . . .                       | 4        |
| 2.2      | Classes vs. Objetos: A Planta e a Casa . . . . .                | 4        |
| 2.3      | Atributos: O que o objeto TEM (Estado) . . . . .                | 4        |
| 2.4      | Métodos: O que o objeto FAZ (Comportamento) . . . . .           | 4        |
| 2.5      | Mão na Massa: Criando nossa primeira Classe . . . . .           | 5        |
| 2.6      | Instanciando: Dando vida ao Objeto . . . . .                    | 5        |
| 2.7      | Exercício de Fixação . . . . .                                  | 5        |
| <b>3</b> | <b>Aula 3: Métodos Construtores e a Palavra-chave 'this'</b>    | <b>7</b> |
| 3.1      | O Problema da Aula Anterior . . . . .                           | 7        |
| 3.2      | O que é um Método Construtor? . . . . .                         | 7        |
| 3.3      | A Palavra-chave <code>this</code> . . . . .                     | 7        |
| 3.4      | Mão na Massa: Evoluindo nosso Carro . . . . .                   | 7        |
| 3.5      | O Construtor Padrão (Default) . . . . .                         | 8        |
| 3.6      | Exercício Prático . . . . .                                     | 8        |

# 1 Aula 1: Primeiros Passos com Java e POO no IntelliJ IDEA

## 1.1 O Ecossistema Java

Antes de escrevermos qualquer código, é crucial entender por que o Java é uma das linguagens mais utilizadas no mundo corporativo e acadêmico. O grande diferencial do Java é o lema *"Write Once, Run Anywhere"* (Escreva uma vez, rode em qualquer lugar). Isso é possível graças a três pilares:

- **JDK (Java Development Kit):** É o kit de ferramentas para nós, desenvolvedores. Contém o compilador e as bibliotecas necessárias para criar programas.
- **JRE (Java Runtime Environment):** É o ambiente de execução. Se um usuário quer apenas rodar um programa Java, ele precisa apenas do JRE.
- **JVM (Java Virtual Machine):** É o coração do Java. Ela traduz o código compilado (chamado de *Bytecode*) para a linguagem específica do sistema operacional (Windows, Linux, macOS).

## 1.2 Preparando o Ambiente: IntelliJ IDEA

Nossas aulas práticas utilizarão o **IntelliJ IDEA** (versão Community), a IDE (Ambiente de Desenvolvimento Integrado) mais popular e robusta para o mercado Java moderno.

1. **Iniciar um Novo Projeto:** Abra o IntelliJ IDEA. Na tela inicial, clique no botão **New Project** (Novo Projeto).
2. **Configurar a Base do Projeto:** No campo **Name**, digite o nome do projeto (ex: Aula01\_P00). Em **Language**, certifique-se de que Java está selecionado. Em **Build system**, mantenha IntelliJ. No campo **JDK**, selecione a versão instalada na sua máquina (recomendado Java 17 ou superior).
3. **Adicionar Código de Exemplo:** Marque a caixa **Add sample code** (Adicionar código de exemplo). Isso fará com que a IDE crie automaticamente o arquivo base do programa para você. Clique em **Create**.
4. **Explorar a Estrutura de Pastas:** No painel esquerdo (Project), expanda a pasta do seu projeto. O código-fonte sempre ficará dentro da pasta amarela chamada `src` (source).

## 1.3 Anatomia do Seu Primeiro Código

Se você marcou "Add sample code", o IntelliJ criou um arquivo chamado `Main.java` com o seguinte conteúdo:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Hello and welcome!");  
4     }  
5 }
```

Vamos dissecar cada linha deste código, pois ele contém a essência do Java:

- `public class Main`: Em Java, **tudo** deve estar dentro de uma Classe. A classe é o bloco de construção fundamental da Programação Orientada a Objetos. O nome do arquivo físico (`Main.java`) deve ser exatamente igual ao nome da classe pública.

- `public static void main(String[] args):` Este é o **Ponto de Entrada** do seu programa. Quando você manda a JVM executar o seu software, ela procura exatamente por essa linha para saber por onde começar.
- `System.out.println("...");` É o comando responsável por imprimir uma mensagem no console e pular para a próxima linha.

**Dica Prática:** Para executar o código no IntelliJ, basta clicar no ícone de **Play (triângulo verde)** que aparece na margem esquerda da tela, ao lado do método `main`, ou usar o atalho `Shift + F10`.

## 1.4 O que é Programação Orientada a Objetos (POO)?

Até agora, programamos de forma procedural. Mas o Java brilha na POO. Na Orientação a Objetos, nós paramos de pensar apenas em "funções soltas" e passamos a modelar o software baseados no mundo real.

Dois conceitos fundamentais que aprofundaremos nas próximas aulas:

1. **Classe:** É a planta-baixa, o molde. (Ex: A planta de uma casa).
2. **Objeto:** É a instância, aquilo que foi construído usando o molde. (Ex: A casa construída na rua X, número 10, pintada de azul).

## 1.5 Exercício de Fixação

1. Apague o comando `System.out.println` original.
2. Crie três linhas de impressão novas para exibir no console: Seu nome completo, o nome do curso e uma frase curta explicando o que é a JVM.
3. Execute o programa e verifique a saída na aba "Run" na parte inferior do IntelliJ.

## 2 Aula 2: Desvendando a Orientação a Objetos

### 2.1 O Paradigma Orientado a Objetos

Até agora, nossos programas eram apenas uma sequência de comandos lidos de cima para baixo. Na Programação Orientada a Objetos (POO), nós mudamos a forma de pensar: passamos a enxergar o software como um conjunto de "coisas" (objetos) que interagem entre si.

Para que isso funcione, precisamos entender quatro conceitos fundamentais: **Classes, Objetos, Atributos e Métodos**.

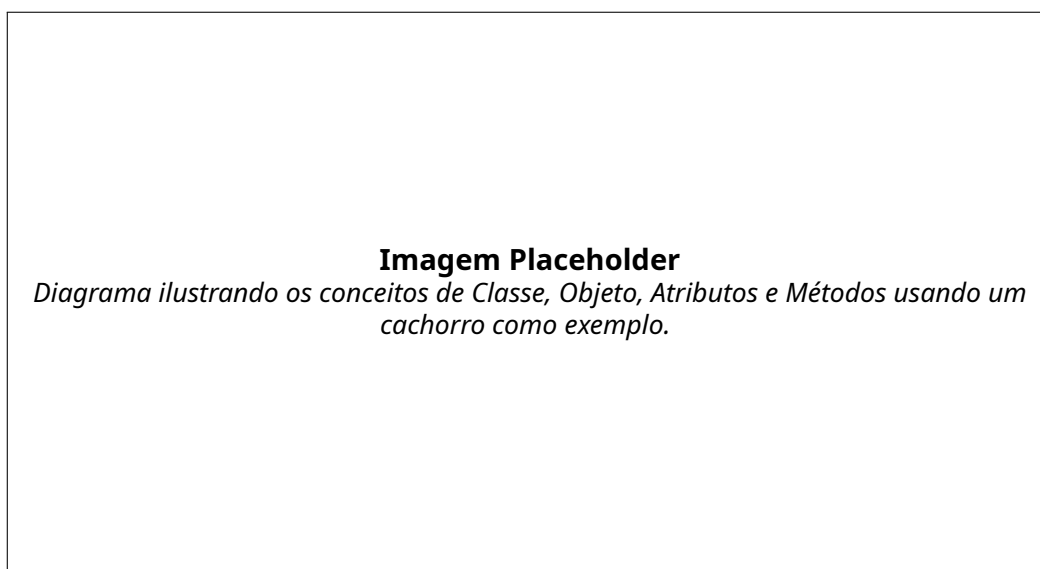


Figura 1: Os 4 pilares práticos da POO.

### 2.2 Classes vs. Objetos: A Planta e a Casa

A maior confusão para quem está começando é entender a diferença entre uma Classe e um Objeto. Pense na construção civil:

- **A Classe é a Planta-Baixa (O Molde):** Ela define onde ficam os quartos, o tamanho da sala e a cor das paredes. Porém, você não pode morar em uma planta-baixa. Ela é apenas um conceito, um projeto. Em Java, a Classe define quais características e ações algo terá.
- **O Objeto é a Casa Pronta (A Instância):** É a casa construída a partir da planta-baixa. Você pode construir várias casas idênticas usando a mesma planta. Cada casa construída é um **Objeto** independente. Em Java, chamamos o ato de criar um objeto de **instanciação**.

### 2.3 Atributos: O que o objeto TEM (Estado)

Os atributos são as características do nosso objeto. Se estivermos criando uma classe `Carro`, quais são as características que todo carro tem? Cor, Modelo, Ano de Fabricação, Velocidade atual. No código Java, os atributos são representados por **variáveis** declaradas dentro da Classe.

### 2.4 Métodos: O que o objeto FAZ (Comportamento)

Os métodos são as ações que o objeto pode executar. O que um `Carro` faz? Acelerar, Frear, Ligar o motor. No código, os métodos são representados por **funções** declaradas dentro da Classe.

## 2.5 Mão na Massa: Criando nossa primeira Classe

Vamos criar nosso molde de um Carro no IntelliJ:

1. No IntelliJ, clique com o botão direito na pasta `src`, vá em **New > Java Class**.
2. Digite Carro (sempre com a primeira letra maiúscula) e aperte Enter.

O código da sua classe Carro ficará assim:

```
1 public class Carro {
2     // 1. Atributos (Características)
3     String modelo;
4     String cor;
5     int ano;
6     int velocidadeAtual = 0;
7
8     // 2. Metodos (Acoes)
9     public void acelerar() {
10         velocidadeAtual = velocidadeAtual + 10;
11         System.out.println("O " + modelo + " acelerou! Velocidade: " +
12             velocidadeAtual + " km/h");
13     }
14 }
```

## 2.6 Instanciando: Dando vida ao Objeto

Abra o seu arquivo `Main.java` e digite o seguinte código:

```
1 public class Main {
2     public static void main(String[] args) {
3
4         // Criando (instanciando) o objeto
5         Carro meuCarro = new Carro();
6
7         // Configurando os atributos
8         meuCarro.modelo = "Fusca";
9         meuCarro.cor = "Azul";
10        meuCarro.ano = 1975;
11
12        // Chamando um metodo
13        meuCarro.acelerar();
14        meuCarro.acelerar();
15    }
16 }
```

**Dica Prática:** Rode o código (Shift + F10). Você verá no console que o Fusca acelerou duas vezes, chegando a 20 km/h. O estado do objeto foi alterado pelas suas próprias ações!

## 2.7 Exercício de Fixação

1. Crie uma nova classe chamada `ContaBancaria`.
2. Adicione os atributos: `titular (String)` e `saldo (double)`.
3. Crie um método `depositar(double valor)` que aumenta o saldo da conta e imprime o novo saldo.

4. Na classe `Main`, crie um objeto da sua conta, defina seu nome como titular, e faça um depósito de R\$ 150.00.

## 3 Aula 3: Métodos Construtores e a Palavra-chave 'this'

### 3.1 O Problema da Aula Anterior

Na Aula 2, nós criamos um `Carro` e depois fomos preenchendo suas características linha por linha:

```
1 Carro meuCarro = new Carro();
2 meuCarro.modelo = "Fusca";
3 meuCarro.cor = "Azul";
4 meuCarro.ano = 1975;
```

Embora funcione, isso gera um problema no mundo real: **Objetos nascendo incompletos**. Imagine que esquecemos de definir o modelo do carro. Teríamos um carro "sem identidade" rodando pelo nosso sistema, o que pode causar erros graves no futuro (o famoso *NullPointerException*).

Como garantimos que um objeto já "nasça" com todas as informações essenciais? É aqui que entram os **Construtores**.

### 3.2 O que é um Método Construtor?

O **Construtor** é um método especial que é executado automaticamente no exato momento em que o objeto é criado (quando usamos a palavra `new`). O objetivo principal dele é **inicializar** o objeto, ou seja, dar valores iniciais para os atributos.

**As Regras de Ouro dos Construtores:**

1. **Mesmo Nome da Classe:** O construtor deve ter exatamente o mesmo nome da classe onde ele está.
2. **Sem Retorno:** Diferente dos métodos normais (como o `acelerar()`), o construtor **não** tem tipo de retorno, nem mesmo `void`.

### 3.3 A Palavra-chave `this`

Quando criamos um construtor, geralmente recebemos parâmetros que vêm de fora para preencher os atributos da classe. Para manter o código limpo e compreensível, é comum dar-mos aos parâmetros o mesmo nome dos atributos. Mas como o Java vai saber quem é quem? Usando o `this`.

- `this` significa **"este objeto"**.
- `this.modelo` refere-se ao atributo que pertence ao objeto.
- `modelo` (sem o `this`) refere-se à variável que chegou como parâmetro.

### 3.4 Mão na Massa: Evoluindo nosso Carro

#### 1. Atualizando a Classe `Carro.java`:

```
1 public class Carro {
2     // Atributos
3     String modelo;
4     String cor;
5     int ano;
6     int velocidadeAtual;
7 }
```

```

8 // METODO CONSTRUTOR
9 public Carro(String modelo, String cor, int ano) {
10     // this.atributo = parametro;
11     this.modelo = modelo;
12     this.cor = cor;
13     this.ano = ano;
14     this.velocidadeAtual = 0; // Todo carro ja nasce parado
15 }
16
17 // Metodo comum
18 public void acelerar() {
19     this.velocidadeAtual += 10;
20     System.out.println("O " + this.modelo + " acelerou! Velocidade: " + this
21     .velocidadeAtual);
22 }

```

**2. Atualizando o Main.java:** Agora, o Java não nos permite mais criar um carro vazio. Somos obrigados a passar os dados no momento da criação!

```

1 public class Main {
2     public static void main(String[] args) {
3
4         // O objeto ja nasce pronto, com todos os dados obrigatorios!
5         Carro meuCarro = new Carro("Fusca", "Azul", 1975);
6
7         meuCarro.acelerar();
8     }
9 }

```

*Note como o código no Main ficou muito mais limpo e seguro (de 4 linhas passou para apenas 1)!*

### 3.5 O Construtor Padrão (Default)

Você deve estar se perguntando: *"Mas na Aula 2 não criamos construtor nenhum e funcionou. Por quê?"*

Quando você não escreve **nenhum** construtor na sua classe, o Java, nos bastidores, cria um construtor invisível e vazio para você. Ele é chamado de **Construtor Padrão**. Porém, **atenção:** a partir do momento em que você escreve o seu próprio construtor (com parâmetros), o Java remove o construtor padrão invisível.

### 3.6 Exercício Prático

Pegue o código da ContaBancaria que criamos na Aula 2 e atualize-o:

1. Crie um construtor na classe ContaBancaria que obrigue o preenchimento do titular e do saldo inicial no momento em que a conta é criada.
2. Use a palavra `this` para diferenciar os atributos dos parâmetros.
3. Vá até a classe Main e modifique a instância da conta para usar o seu novo construtor, passando seu nome e 500.0 como saldo inicial.